



Lesson 6: k-NN, Naive Bayes and Support Vector Machines

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi dell'Aquila

Before we start

- Exercise 5, which we discuss now
- The score that was too good, and what the honest number cost you

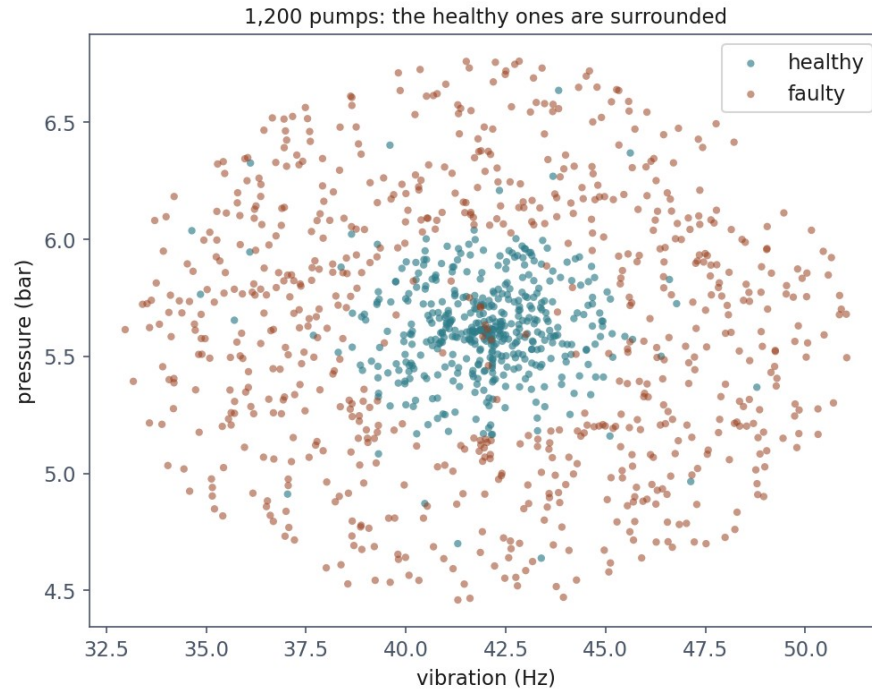
Today: the first lesson that offers a choice

- **k-nearest neighbours (k-NN)**: remember, and vote
- The **curse of dimensionality**: the bill for that
- **Naive Bayes**: one strong assumption, bought cheap
- **Support vector machines (SVM)**: margins, and kernels
- Which family to reach for, and why that beats tuning

1,200 pumps, two readings each

- Vibration in Hz, pressure in bar
- A pump is faulty when its readings fall **outside the design envelope**, too low as readily as too high
- **61.3%** faulty; 4% of the labels are deliberately flipped

The shape of the problem



What a straight boundary costs

Model	Cross-validated accuracy
Always predict the majority class	0.613
Logistic regression (lesson 4)	0.613 $\pm$ 0.000

k-nearest neighbours: the whole algorithm

- To classify a new point, find the **k closest** training points and take the majority vote
- There is no training step: “fitting” means **storing the data**
- Called a **lazy learner**: an unusually honest name for an algorithm

Distance is all it has

$$d(x, x') = \sqrt{\sum_{j=1}^n (x_j - x'_j)^2}$$

Scale first, or vibration decides everything

- Vibration runs over **tens of Hz**; pressure over **a couple of bar**
- Unscaled, a one-bar pressure difference is invisible beside a ten-Hz one
- Lesson 2's scaling argument, arriving with **immediate consequences**

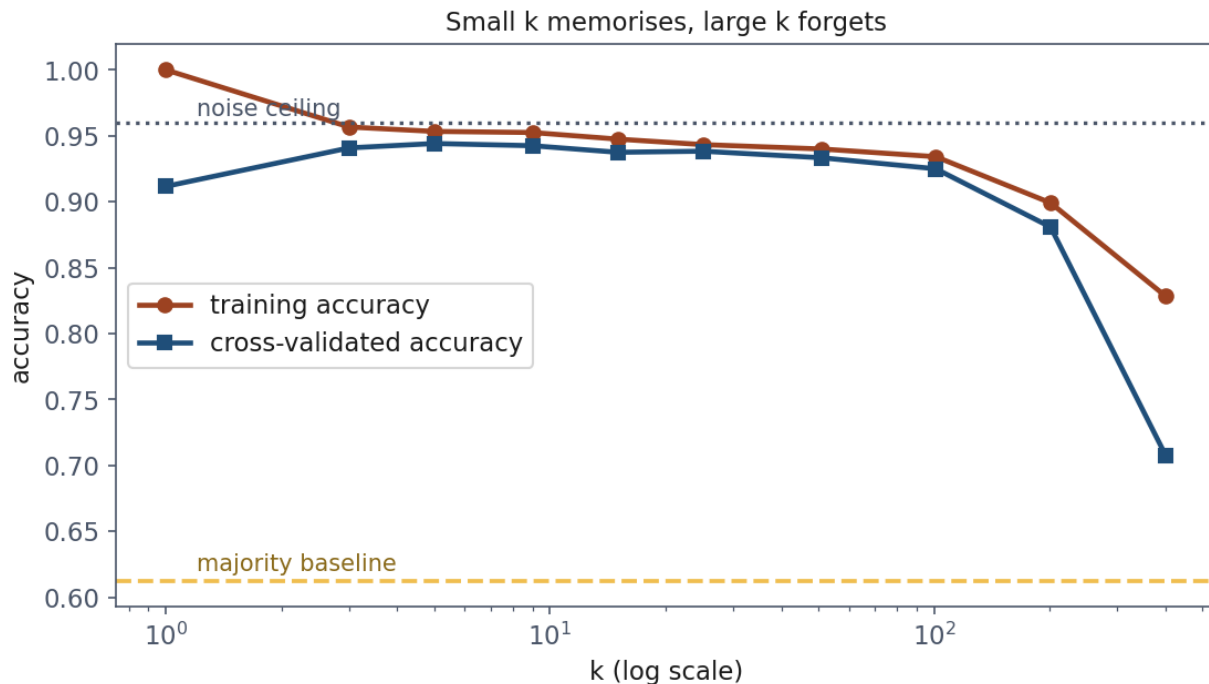
k is the bias-variance dial, made visible

- **Small k**: the boundary follows every point, mislabelled ones included. Low bias, high variance
- **Large k**: the vote is taken over a wide neighbourhood, and eventually stops following real structure. High bias, low variance
- At $k = 1$ the training accuracy is **exactly 1.000**. Always, on any dataset

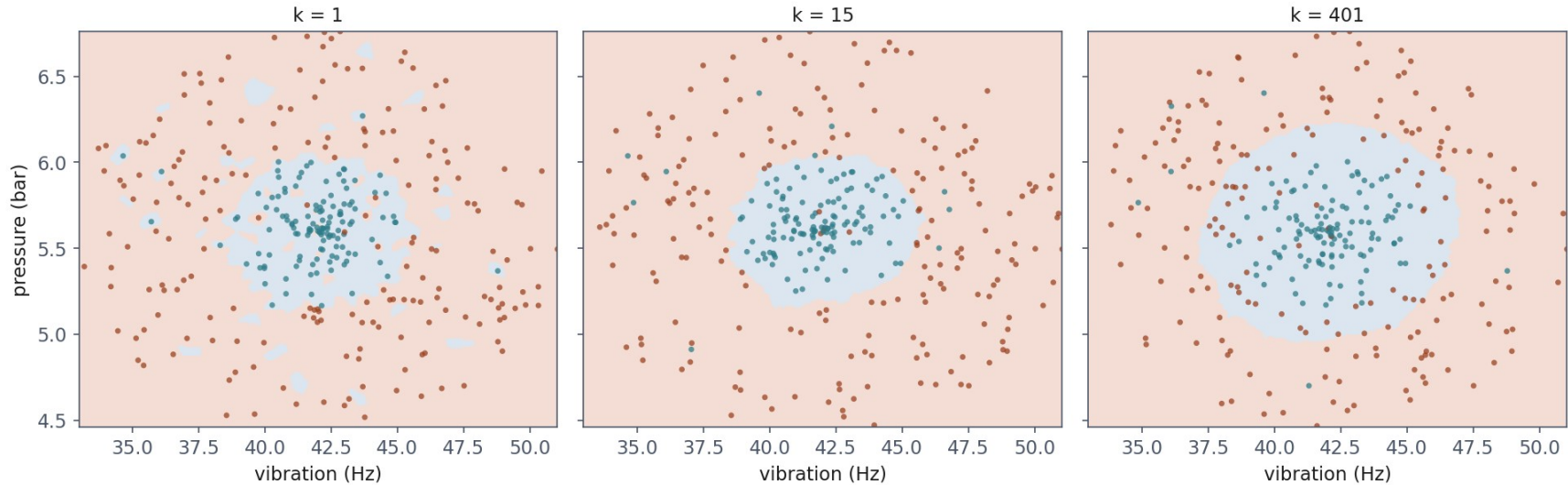
Measured on the pumps

k	Training	Cross-validated
1	1.000	0.912
5	0.953	0.944
15	0.948	0.938
51	0.940	0.933
401	0.828	0.708

The two curves, and the two lines that bound them



The same data at three values of k



What k-NN costs

- No training time at all: you pay at **every prediction** instead
- A naive implementation compares the query against **every** training row: $O(mn)$ for m rows and n features
- **The model is the dataset.** You cannot ship one without the other

Now the price: add columns of nothing

- Extra columns of **pure noise**, drawn from a normal distribution
- The two real readings are **untouched**: the problem is exactly as solvable
- Only the number of columns changes

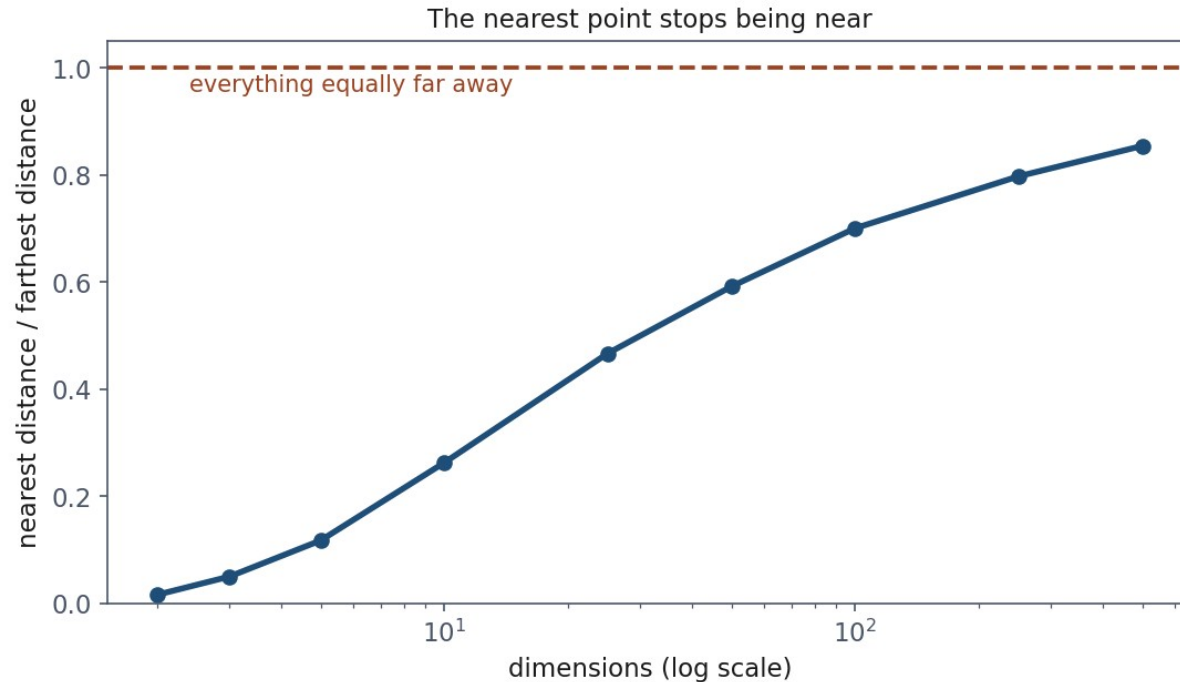
The signal never left

Total columns	Accuracy	Above baseline
2	0.938	+0.325
12	0.762	+0.149
27	0.662	+0.049
52	0.602	-0.011
102	0.578	-0.035

Why: distances stop varying

- Scatter points in a cube, pick one, measure its **nearest** and its **farthest** neighbour
- In two dimensions those are very different numbers, which is what makes “nearest” a meaningful word
- Each new dimension adds its own squared difference, and the contributions **average out**

The nearest point stops being near



Nearest divided by farthest

Dimensions	nearest ÷ farthest
2	0.016
10	0.263
50	0.592
100	0.701
500	0.855

In 100 dimensions the nearest point is 70% as far as the farthest

- In **two** dimensions it is **2%**: “nearest” picks out something special
- In **one hundred** it is **70%**: the nearest point is barely nearer than a random one
- A vote among “the five nearest” becomes a vote among five taken at random

What the curse is, and is not

- **Not** that high-dimensional problems are unlearnable: lesson 9's networks work in thousands of dimensions
- That **methods built on distance lose their footing**, because the quantity they depend on stops varying
- k-NN, k-means (lesson 8), and radial basis function (RBF) kernels, all of them

Notebook 1, live

- k-NN on the pumps, choosing k, and the curse measured

Break

- Twelve minutes

Naive Bayes: turn the question around

- We want the probability of the class **given** the readings
- The prior is a count; the denominator is identical across classes and cancels
- Everything hard sits in one term: the probability of **this exact combination of readings** among examples of that class

Bayes' rule, applied to a class

$$P(y = c \mid x) = \frac{P(x \mid y = c) P(y = c)}{P(x)}$$

The assumption: independent, given the class

- **Given the class, the features are independent of one another**
- The joint density then factorises into n one-dimensional densities
- Each factor is estimated from the rows of that class alone

The factorisation the assumption
buys

$$P(x \mid y = c) = \prod_{j=1}^n P(x_j \mid y = c)$$

A wonderful bargain, and almost never true

- Training is **a single pass**; adding features costs almost nothing
- But vibration and pressure are both driven by the operating point
- “New” is not independent of “York” given the topic. Symptoms co-occur
- So the question is not whether it holds: it is **when being wrong about it costs you nothing**

On the pumps, it scores 0.933

Model	Accuracy
Majority baseline	0.613
Logistic regression	0.613
Gaussian Naive Bayes	0.933
k-NN, $k = 5$	0.944

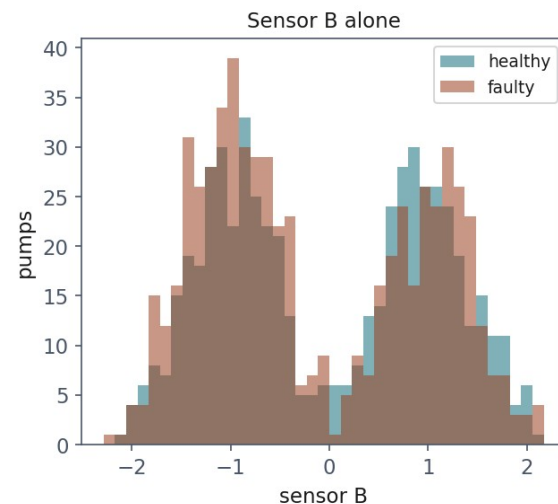
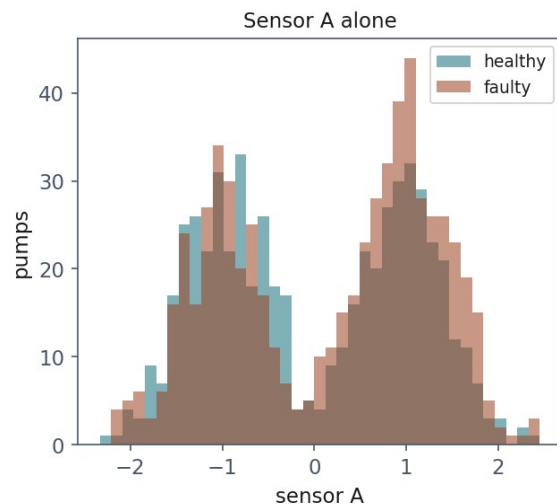
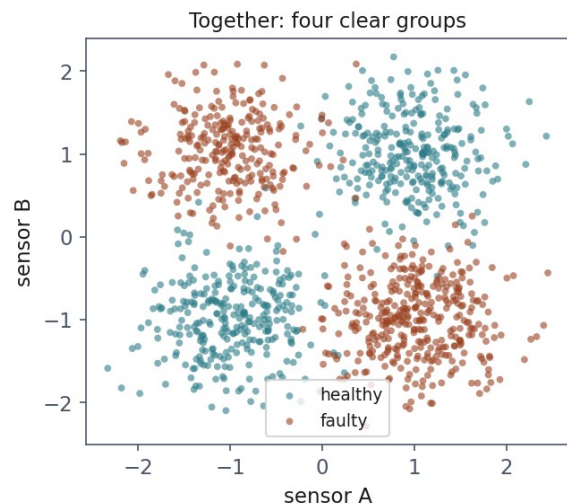
Because here the assumption is true

Correlation between the two readings	Value
overall	-0.046
within healthy pumps	-0.006
within faulty pumps	-0.049

One step away: when the signal is an interaction

- A second pair of sensors on the same fleet
- Faulty when **exactly one** of the two readings is high
- Both high is the designed high-load mode; both low is idle; one without the other is a mismatch between demand and delivery

What Naive Bayes gets to see



0.404: below the baseline, and below chance

Model	Accuracy
Majority baseline	0.523
Gaussian Naive Bayes	0.404
Logistic regression	0.393
SVM, linear kernel	0.606
k-NN, k = 5	0.967
SVM, RBF kernel	0.972

Its probabilities are not probabilities

- Mean confidence when **correct: 0.567**. When **wrong: 0.555**
- It cannot tell the difference between the two situations
- With correlated features it fails the other way: 0.999 reported, with an accuracy nothing like that
- **The ranking may be useful while the probabilities are not**

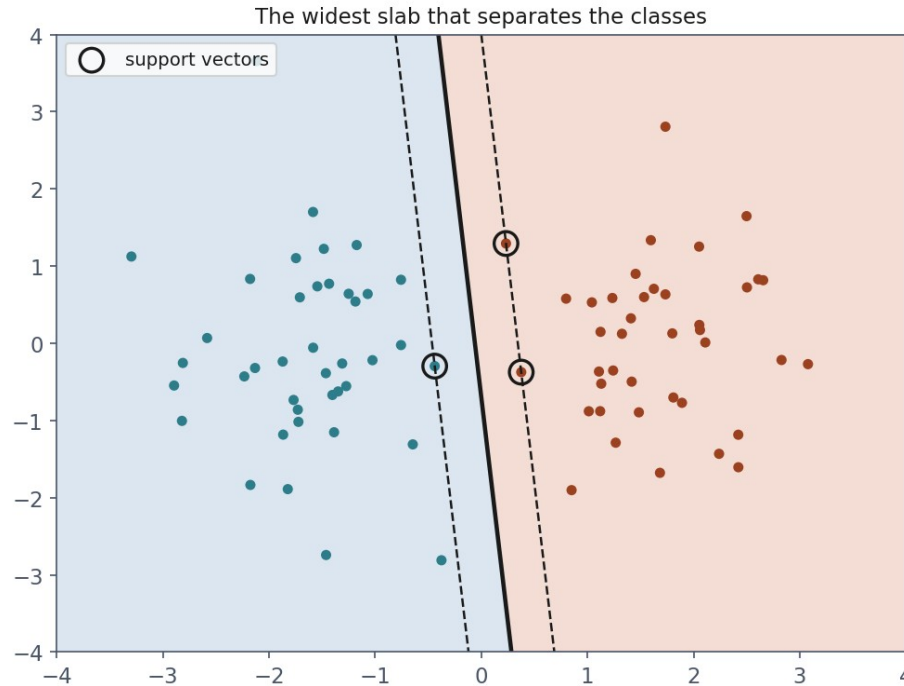
Notebook 2, live

- Where the assumption holds, and one step away where it does not

The margin: a criterion that is not the error

- On separable data there are **infinitely many** separating lines, and every one has **zero training error**
- Minimising the error therefore cannot choose between them
- The SVM picks the widest **slab**: push it out until it touches the nearest point of each class
- A boundary passing close to a point is one small perturbation from getting it wrong

Three points decide everything



Not separable? Charge for the violations

- Our labels are 4% flipped, so the strict problem has **no solution at all**
- Let each point violate the margin by ξ_i , and charge C for every unit of it
- The constraint says: be on the correct side, or pay

A wide slab, minus what the violations cost

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{s. t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i$$

C is the price of a training error

- **Large C**: violations are expensive, so the model contorts to classify everything. Narrow margin, low bias, high variance
- **Small C**: a wider, calmer boundary, at the cost of some errors
- The same dial as k in k -NN and λ in lesson 3, in a third costume
- Note the direction: **large C means less regularisation**

The best straight line is still a straight line

Model	Accuracy	Support vectors
SVM, linear kernel	0.613 $\pm$ 0.000	947 of 1,200 (79%)
SVM, RBF kernel	0.947 $\pm$ 0.005	278 of 1,200 (23%)

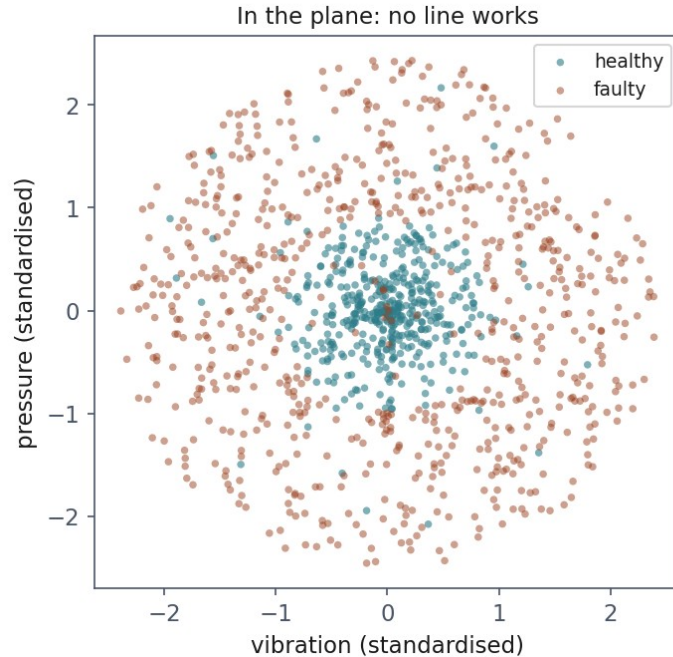
A high support-vector fraction is a free warning

- The linear model needs **79%** of the training set to define its boundary
- The RBF model needs **23%**
- Almost every point sits on or inside the margin: there is no slab that separates anything
- The count falls out of `fit` at no cost, and you should look at it

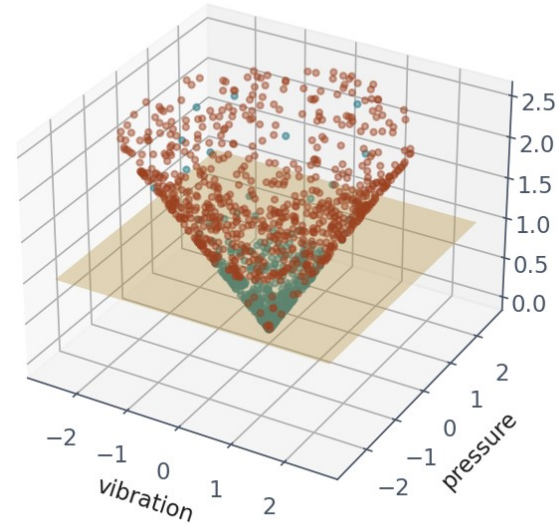
The kernel trick: they needed different coordinates

- Healthy pumps in a disc, faulty ones around it: no line works **in the plane**
- Add a third coordinate: the **distance from the design point**
- Healthy pumps rise a little, faulty ones a lot, and a flat plane separates them perfectly
- The classes were always separable. They were measured in the wrong coordinates

The same pumps, lifted



Lifted: a flat plane does
height = distance from the design point



And the map is never computed

- The solution depends on the data **only through inner products** between pairs of points
- So we need a function returning the inner product **in the new space**, while computing only with the original one
- The radial basis function is scikit-learn's default: one exponential per pair, for an **infinite-dimensional** space

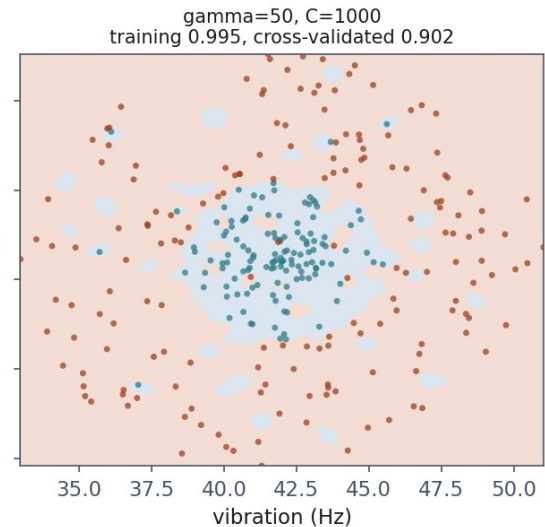
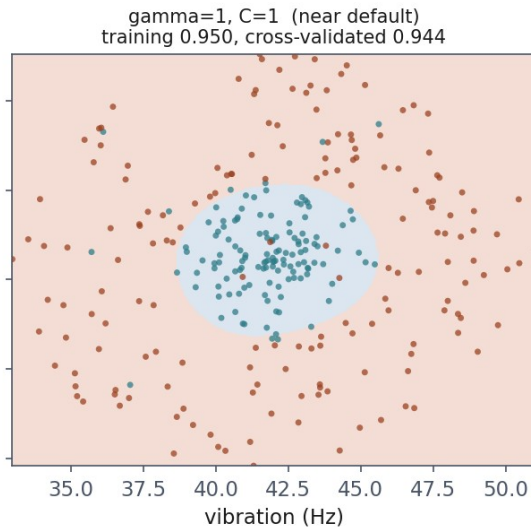
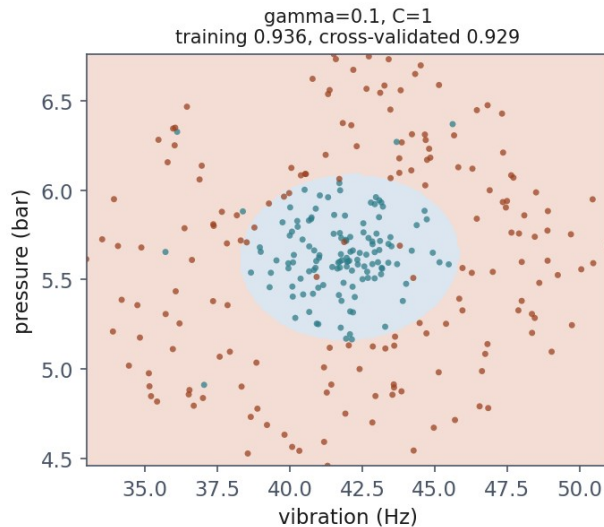
The radial basis function, written out

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

The highest training score, the lowest honest one

Setting	Training	Cross-validated
$\gamma = 0.1, C = 1$	0.936	0.929
$\gamma = 1, C = 1$	0.950	0.944
$\gamma = 50, C = 1000$	0.995	0.902

Overfitting you can see



Notebook 3, live

- Margins, support vectors, and what γ does to the boundary

The three compared, on identical data

Model	Accuracy
Majority baseline	0.613
Logistic regression (lesson 4)	0.613
SVM, linear kernel	0.613
Gaussian Naive Bayes	0.933
k-NN, k = 5	0.944
SVM, RBF kernel	0.947
<i>noise ceiling</i>	≈ 0.96

How to choose, in practice

Situation	Reach for
Few features, plenty of data, odd-shaped boundary	k-NN, or an RBF SVM
Very many features, little data	Naive Bayes
Need a calibrated probability	Not Naive Bayes
Model must be small, or fast at prediction	SVM, not k-NN
The signal is an interaction between features	k-NN or a kernel

What to take away

- **No line separates the pumps:** 0.613, the base rate
- **k is the bias-variance dial**, made visible
- **In 100 dimensions the nearest point is 70% as far as the farthest**
- **Naive Bayes:** 0.933 where independence holds, **0.404** where it does not
- **Choosing the family beats tuning the wrong one:** 0.613 to 0.947

Homework

- **Exercise 6**, discussed at the start of **Friday 6 November**
- A dataset, three families, and a defence of which one you chose